NutriVision

Food Nutrition Estimator

Developer Guide & Design Documentation

1. Project Overview

NutriVision is a web-based application that estimates the nutritional content of Indian food from a single photo. The user uploads a picture of their meal with a Rs. 10 coin placed flat in the frame, and the system returns the food names, estimated weights, and a full nutrition breakdown including calories, protein, carbs, fat, and micronutrients.

The system works end to end without any manual input beyond the photo. The coin acts as a physical size reference, so no knowledge of camera specs or distance is needed.

Component	Technology Used
Segmentation	SAM3 (facebook/sam3) via HuggingFace Transformers
Classification	ConvNeXtV2-Tiny (fine-tuned on 80 Indian food classes)
Geometry Estimation	PCA (scikit-learn) on segmentation masks
Scale Reference	Rs. 10 coin (diameter = 2.7 cm)
Weight Estimation	LLM (Qwen via Ollama Cloud)
Nutrition Lookup	Custom JSON database (79 dishes, per 100g)
Frontend	Gradio 6.x
Hosting	Lightning AI (GPU Studio)

---

2. Design Journey — How We Got Here

This section documents the thinking and decisions made at each stage of building the pipeline. Some of these approaches were tried and then changed; understanding why helps explain the final design.

2.1 The First Idea: YOLO for Detection, then SAM3

Our original plan was to use a YOLO model for multi-food detection. The idea was simple: run YOLO on the image to get bounding boxes for each food item, then feed those boxes as prompts to SAM3 to get clean segmentation masks.

This sounds straightforward but ran into a real problem in practice. Food detection from a single overhead photo is genuinely hard. Dishes overlap, lighting is inconsistent, and many Indian foods look similar to each other (think biryani vs. khichdi from above). YOLO models trained on standard food datasets had low accuracy when tested on real meal photos — the kind someone would actually take at home or in a restaurant.

Why YOLO failed here

Most food detection datasets are built around isolated dishes on white backgrounds or restaurant-style photographs. Real-life meal photos — uneven lighting, partially eaten food, dishes touching each other — look very different. A YOLO model would miss items or produce overlapping boxes that did not correspond to actual individual servings.

Because the YOLO detection step was unreliable, the boxes fed into SAM3 were often wrong, and the final masks were noisy or incomplete. The whole pipeline quality depended on that first step, and it was the weakest link.

2.2 The Fix: Segment First, Classify Later

The insight that changed everything was flipping the order. Instead of detecting food and then segmenting, we do segmentation first — using SAM3's built-in text prompt capability directly on the original image — and then classify what we have.

SAM3 is a segment-anything model that can take natural language prompts. We run it three times with different prompts to catch different types of food presentation:

- Prompt 1: "food and its bowl" — catches dishes served in bowls or plates with a visible container
- Prompt 2: "drink and its glass" — catches beverages
- Prompt 3: "food" — a generic fallback to catch anything missed above
- Coin prompt: "coin" — runs separately to find the size reference

After collecting all detections from all three prompts, we merge them. Detections with high mask overlap (IoU above 0.40) are treated as duplicates and only the best one is kept. This merge step also handles a common problem: if a food item is detected by both the specific prompt ("food and its bowl") and the generic prompt ("food"), we prefer the specific one because it tends to give a better mask boundary.

Key insight

SAM3 is much better at segmenting objects when told what to look for than a dedicated detector is at finding food in the wild. By trusting SAM3 for segmentation and using ConvNeXtV2 for the "what is this?" question, each model does what it is actually good at.

2.3 Classification with ConvNeXtV2

Once we have a clean mask for a food region, we crop that region from the original image, fill the non-food pixels with a neutral grey (RGB 128), and pass it to a fine-tuned ConvNeXtV2-Tiny model. The grey fill is important — without it, adjacent food items would bleed into the crop and confuse the classifier.

The model was trained on 80 Indian food classes. If the top predicted class scores below 40% confidence, we discard that detection. This threshold was chosen to balance catching real items against creating false detections from background clutter.

2.4 The Scale Problem: How Do We Know Real-World Size?

Estimating how much food weighs requires knowing how large it actually is in the real world — not just in pixels. The number of pixels an object occupies depends on how far away the camera was, the focal length of the lens, and the sensor size. None of these are easy to get from a phone photo without special metadata.

We initially considered using monocular depth estimation. A depth model can estimate the distance of each pixel from the camera based on visual cues like perspective, shading, and known object sizes. With depth, plus the camera's intrinsic parameters (focal lengths fx and fy), you can convert pixel distances into real-world centimetres.

Approach	What it needs	Problem
Monocular depth estimation	fx, fy (focal length), depth map	Camera intrinsics not available in a web app. Only possible with ARKit on iPhone.
Rs. 10 coin as reference	Coin visible in photo (diameter = 2.7 cm)	Works with any camera. User just places a coin in frame.

The depth approach could work with ARKit on iOS because iPhones expose their camera intrinsics and LiDAR data through the ARKit API. But since this project is a web app accessible on any device, that was not a viable option.

The coin solution is simple and it works. A Rs. 10 RBI coin has a known diameter of 2.7 cm. We detect the coin using SAM3, run PCA on its mask, and measure how many pixels correspond to 2.7 cm. This gives us a pixels-per-centimetre ratio that we then apply to all food items in the same image. The assumption is that everything is on roughly the same flat surface — which is true for a plate of food on a table.

2.5 Measuring Food Dimensions with PCA

With a pixels-per-centimetre scale in hand, we need to measure the size of each food item. We use Principal Component Analysis (PCA) for this.

For each segmented food region, we collect all the (x, y) pixel coordinates that belong to the mask. PCA finds the two directions of maximum variance in this point cloud. The first principal component (PC1) points along the longest axis of the shape, and the second component (PC2)

points along the shortest axis. The spread of the data along each component gives us the approximate width and height of the food.

Specifically, we use 4 standard deviations along each axis (2 on each side of the centre) as a reasonable estimate of the full extent of the shape. This corresponds to drawing an ellipse around the food region. We then convert from pixels to centimetres using the coin scale.

What PCA gives us

Two measurements: a major axis (the longer dimension, which we label as width W) and a minor axis (the shorter dimension, labelled height H). These are visualised as a red and green arrow overlaid on the food crop image. The LLM uses these labelled images to reason about the food geometry.

2.6 Weight Estimation: Why an LLM instead of an MLP

Estimating weight from dimensions requires knowing the geometry of the food. A sphere of radius R has volume (4/3) x pi x R^3. A flat disc like a roti can be approximated as a cylinder. A piece of chicken has a more complex shape. The right formula depends on the food item.

The textbook approach for this kind of regression is to train a small neural network (an MLP) that takes the dimensions and food class as input and outputs a weight estimate. We considered this, but training an MLP well requires a large dataset of (image, actual weight) pairs — ideally thousands of examples for each food class. We did not have that data.

Instead, we use an LLM (Qwen via Ollama Cloud). The LLM receives:

- The food class name
- The major and minor axis measurements in centimetres
- The PCA annotated image showing the ellipse overlay
- The density value for this food (looked up from a hand-curated JSON database)
- A prompt that asks it to choose a suitable geometric shape, compute volume, and multiply by density

The LLM has broad knowledge of food shapes and common geometric models. It can reason that a samosa is roughly a tetrahedron, that idli is a hemisphere, that a chapati is a flat disc. This kind of qualitative shape knowledge is exactly what an MLP cannot learn without seeing thousands of labelled examples.

Accuracy in practice

For clear photos where both axes are visible, the LLM weight estimation is typically within 30 grams of the actual weight. Results degrade for complex overlapping dishes or when the coin is partially obscured.

2.7 Why Density is Not Left to the LLM

One important detail: the density value is never estimated by the LLM. It is looked up from a hand-curated food_density.json file and passed explicitly in the prompt. The LLM is instructed to copy the density value verbatim and not change it.

The reason for this is reliability. LLMs can hallucinate plausible-sounding but incorrect numerical values. Food density is a specific physical constant that depends on how the food is prepared and served — values like 0.53 g/cm3 for idli (published in food science literature) or 1.25 g/cm3 for jalebi (heavy sugar syrup saturation) are not things we want to leave to guesswork. After the LLM returns its estimate, we overwrite the density field with the correct JSON value as a hard enforcement step.

---

3. The Final Pipeline in Detail

This section walks through the complete pipeline as it runs in production, step by step.

Step 1 — Image Preprocessing

The uploaded image is resized to 1008 x 1008 pixels before any GPU work. This is SAM3's standard square input size. Phone photos are often 3000-4000+ pixels on a side and would exhaust GPU memory if fed in at full resolution. The resize uses Lanczos resampling to preserve visual quality. Small images are upscaled rather than rejected.

Step 2 — SAM3 Segmentation (Four Passes)

SAM3 runs four separate times on the resized image:

1. Pass 1 with prompt "food and its bowl" — targets dishes served in containers
2. Pass 2 with prompt "drink and its glass" — targets beverages
3. Pass 3 with prompt "food" — generic fallback for anything missed
4. Pass 4 with prompt "coin" — separate pass just for the scale reference coin

After each pass the CUDA cache is cleared to avoid memory buildup across the four GPU calls.

Each pass returns a list of masks, bounding boxes, and confidence scores. We apply a confidence threshold of 0.60 to keep only high-quality detections.

Step 3 — Merging Detections

The three food detection passes can return overlapping detections for the same item. The merge step deduplicates them:

5. Compute pairwise mask IoU (intersection over union) for every pair of detections across all three passes
6. Build groups of detections where any two members have IoU >= 0.40
7. For each group, keep the detection from the most specific prompt ("food and its bowl" beats "food"). If tied, keep the one with the highest confidence score
8. Remove food-only detections that are heavily contained within a container detection (containment threshold 0.25)

This merge step ensures that a bowl of biryani, for example, is represented by a single clean mask of the bowl+contents rather than two overlapping masks from different prompts.

Step 4 — Coin Scale Extraction

The best coin detection (highest confidence) is selected from the coin pass. PCA is run on the coin mask to find the major axis, which represents the true diameter of the coin. The scale factor in pixels per centimetre is:

```
pixels_per_cm = major_axis_pixels / 2.7
```

If no coin is detected, the pipeline raises a CoinNotFoundError and stops immediately. There is no fallback scale — without the coin, any weight estimate would be meaningless. The user sees a clear message asking them to add the coin and retry.

Step 5 — Food Classification

For each merged food detection, we:

9. Crop the bounding box region from the resized image
10. Within the crop, set all pixels outside the mask to RGB (128, 128, 128) — neutral grey
11. Pass the masked crop through the ConvNeXtV2-Tiny classifier
12. Apply softmax to get class probabilities
13. If the top class probability is below 0.40, discard this detection entirely

The grey fill is a small but important detail. Without it, adjacent food items would appear in the crop and confuse the classifier. The model sees only the masked region against a uniform neutral background.

Step 6 — PCA Geometry

For each classified food item, we run PCA on the mask pixels to compute:

- Center point (cx, cy) of the food region
- Major axis length in pixels (4 standard deviations along PC1)
- Minor axis length in pixels (4 standard deviations along PC2)
- Rotation angle of the major axis

We convert the pixel measurements to centimetres using the coin scale. The result is two numbers: major_cm and minor_cm.

We then generate a PCA visualisation image: the original crop with the mask highlighted in blue, a dashed ellipse showing the PCA fit, and two annotated arrows — red for the major axis labelled "W=X cm" and green for the minor axis labelled "H=X cm". This image is what gets sent to the LLM.

Step 7 — LLM Weight Estimation

Each food item gets its own LLM call. The prompt contains:

- The food class name
- major_cm and minor_cm as plain numbers
- The PCA image (base64 encoded, sent as a vision input)
- The density value from food_density.json (4 decimal places)
- An explicit instruction to choose a geometric model, compute volume, and output weight_g = volume x density

The LLM responds with a structured JSON object (enforced by Pydantic parsing) containing weight_g, geometry (the shape chosen), density_g_per_cm3, and reasoning (one sentence explanation). After parsing, the code overwrites density_g_per_cm3 with the authoritative database value regardless of what the LLM returned.

If the LLM response fails to parse, the weight defaults to 100 g and the reasoning field notes the parse error. This prevents the pipeline from crashing on a bad LLM response.

Step 8 — Nutrition Scaling

With an estimated weight in grams, we look up the nutrition data for the food class from food_nutrition.json. The database stores values per 100 g, so all nutrients are scaled by (weight_g / 100):

- Calories (kcal)

- Carbohydrates, Protein, Fat, Free Sugar, Fibre (all in grams)
- Sodium, Calcium, Iron (mg), Vitamin C (mg), Folate (micrograms)

Step 9 — Combined Annotated Output

The final annotated image overlays all detected food regions on the original (resized) image. Each region gets a semi-transparent colour overlay and a label showing the food name and estimated weight. This is the image the user sees in the Gradio UI.

---

4. Code Structure

File	What it does
app.py	Gradio UI definition. Handles user uploads, streams progress updates, builds the nutrition table, and shows the annotated result image.
pipeline.py	Core logic. Contains all pipeline steps, SAM3 wrappers, PCA computation, LLM calls, and the run_pipeline_steps() generator.
models.py	Lazy model loading. SAM3 and ConvNeXtV2 are loaded once on the first call and cached in module-level globals.
config.py	All constants and paths. SAM3 prompts, confidence thresholds, coin diameter, LLM endpoint, nutrient column names.
files_models/food_nutrition.json	Nutrition database for 80 Indian food classes. Values per 100g. 45 entries from published Indian food composition tables, 35 researched from web sources.
files_models/food_density.json	Density database for 80 food classes. Values in g/cm3 for the cooked/served item including absorbed oil, syrup, or moisture.

---

5. Known Limitations

Coin must be clearly visible

The entire pipeline stops if the coin is not detected. Partially occluded coins, tilted coins, or coins covered by food will cause the pipeline to fail. The system checks for this and gives a clear error message, but there is no graceful fallback.

Depth assumption

We assume all food items are at roughly the same distance from the camera as the coin (i.e., they are all on the same flat surface). If a tall glass is photographed from a slight angle, its apparent size in pixels will be larger than a flat item at the same real-world distance, and the weight estimate will be off.

30g weight error window

For well-framed photos with both axes clearly visible and the coin unobstructed, the typical weight error is under 30 grams. This is acceptable for nutrition tracking but not precise enough for medical dietary management. Complex overlapping dishes and non-standard serving styles increase the error.

79 food classes only

The classifier and the nutrition/density databases only cover 80 Indian food dishes. Foods outside this set will either be misclassified as the closest match or rejected below the confidence threshold. The system does not have an "unknown food" output.

LLM latency

Each food item requires a separate LLM call to the Ollama Cloud API. A plate with three food items will make three sequential calls. This can add significant latency depending on API response time, and it introduces a dependency on an external paid service.

---

6. Future Work

- Train a dedicated MLP or lightweight regression model for weight estimation once a labelled dataset of (food, major_cm, minor_cm, actual_weight_g) pairs is collected. This would eliminate the LLM dependency and reduce latency significantly.
- Add ARKit support for iOS users to replace the coin with depth-based scale estimation using the phone's LiDAR sensor and camera intrinsics.
- Expand the food class coverage beyond 80 dishes, particularly for regional Indian foods that are currently not in the database.
- Implement a confidence-weighted ensemble for classification: run multiple crops at different scales and combine predictions to handle partially obscured food items.
- Add portion tracking across meals — the Gradio app currently handles one photo at a time with no memory between sessions.

---

7. Quick Setup Reference

Environment requirements:

- Python 3.10+, CUDA-capable GPU recommended (T4 or better)
- HuggingFace token with access to facebook/sam3
- Ollama Cloud API key
- ConvNeXtV2 checkpoint file (best_convnextv2_tiny.pt)

Key environment variables:

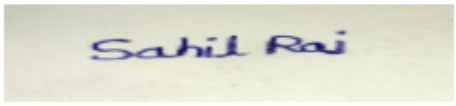
```
export OLLAMA="your_api_key"   # Ollama Cloud key
export HF_TOKEN="your_hf_token"   # HuggingFace token for SAM3
```

Run command:

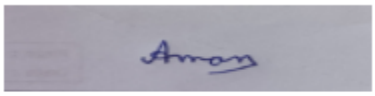
```
python app.py
```

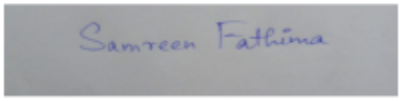
The Gradio UI will be accessible at http://0.0.0.0:7868. On Lightning AI, open port 7868 from the Ports tab.

Reviewed and Approved by:


Sahil Raj

Sahil Sharma


Aman Mani Tiwari


Samreen Fathima

Tasneem Shahnawaz